Assignment 5 (Bonus)

Image Generation Using Diffusion Models

Deep Learning — Spring 2025

Neeka Javed MSDS25008

1. Introduction

This report documents the implementation and evaluation of a Denoising Diffusion Probabilistic Model (DDPM) for image generation. The model was trained on a subset of an animal dataset comprising 5 classes with 20 images per class (100 images total). The objective was to implement the full forward and reverse diffusion pipeline using PyTorch and generate novel images from pure Gaussian noise.

2. Dataset and Preprocessing

Five animal classes were selected from the provided dataset. Each class contributed 20 images, yielding 100 training images in total. The following preprocessing transformations were applied before feeding images into the diffusion process:

- Resize to 64x64 pixels
- Random horizontal flip for basic augmentation
- Convert to tensor
- Normalize to [-1, 1] range using mean=[0.5, 0.5, 0.5] and std=[0.5, 0.5, 0.5]

Normalization to [-1, 1] is critical for diffusion models as the noise schedule operates in this range. The small dataset size (100 images) was the primary constraint on generation quality.

3. Model Architecture

3.1 U-Net Denoising Network

A U-Net architecture was selected as the denoising backbone, consistent with the original DDPM paper. The network takes a noisy image x_t and timestep t as inputs and predicts the noise epsilon that was added at that step.

Component	Details
Encoder Block 1	Conv2d(3, 64) + GroupNorm(32,64) +RELU x2
Encoder Block 2	Conv2d(64, 128) + GroupNorm(32,128) + RELU x2
Bottleneck	Conv2d(128, 256) + GroupNorm(32,256) + RELU x2
Time Embedding	Sinusoidal + MLP(256 -> 1024 -> 256), added at bottleneck
Decoder Block 1	ConvTranspose2d(256,128) + DoubleConv(256,128) + Skip
Decoder Block 2	ConvTranspose2d(128,64) + DoubleConv(128,64) + Skip
Output	Conv2d(64, 3, kernel=1)

3.2 Key Design Choices

- GroupNorm over BatchNorm: GroupNorm is independent of batch size, which is important for small batches common in diffusion training. BatchNorm's batch statistics become unreliable at small batch sizes.
- SiLU (Swish) activation: SiLU is smooth, non-monotonic, and has shown superior performance over ReLU in diffusion models due to better gradient flow.
- Skip connections: Preserve spatial information lost during downsampling, enabling fine-grained noise prediction.
- ConvTranspose2d for upsampling: Learnable upsampling that recovers spatial resolution during decoding.
- Sinusoidal time embedding: Encodes timestep using fixed sinusoidal frequencies, allowing the model to generalize across timesteps without overfitting to a lookup table.

4. Diffusion Process

4.1 Forward Process

The forward process gradually corrupts a clean image x_0 by adding Gaussian noise over T=1000 timesteps. Rather than applying noise directly, the closed-form equation is used to jump to any timestep t in a single step:

```
x_t = sqrt(alpha_bar_t) * x_0  +  sqrt(1 - alpha_bar_t) * epsilon,
                        epsilon ~ N(0, I)
```

A linear beta schedule was used with beta_start=0.0001 and beta_end=0.02. The cumulative product alpha_bar_t controls the noise level at each step.

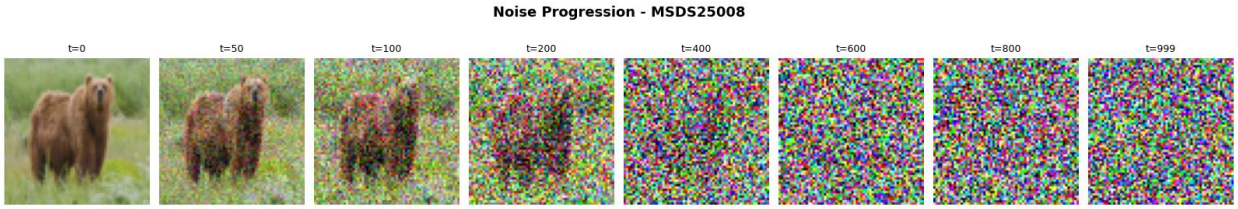

Figure 1 -- Forward noise progression at t = 0, 50, 100, 200, 400, 600, 800, 999. The original bear image is gradually destroyed. By t = 400 most structure is lost, and at t = 999 the image is indistinguishable from pure Gaussian noise.

4.2 Reverse Process

During inference, the model starts from pure Gaussian noise x_T and iteratively denoises using the learned posterior. At each step t, the model predicts the noise epsilon_theta and computes the denoised mean. Small Gaussian noise is added for t > 1 using the posterior variance to maintain stochasticity.

5. Training

5.1 Configuration

Hyperparameter	Value
Optimizer	Adam (lr = 1e-4)
Loss Function	Custom MSE: mean((pred - target)^2)
Batch Size	16
Max Epochs	200
Early Stopping Patience	20 epochs
Noise Steps (T)	1000
Beta Schedule	Linear (1e-4 to 0.02)
Image Resolution	64 x 64

5.2 Loss Function

A custom MSE loss was implemented to measure the difference between the predicted noise and the actual noise added during the forward process. This is the standard DDPM training objective:

```
L = mean( (epsilon - epsilon_theta(x_t, t))^2 )
```

5.3 Training Loss

Training ran for 108 epochs before early stopping triggered. The loss decreased from 0.9422 at epoch 1 to 0.0316 at the best epoch (epoch 88), a reduction of 0.9106. The loss curve shows rapid convergence in the first 30 epochs followed by slower refinement.

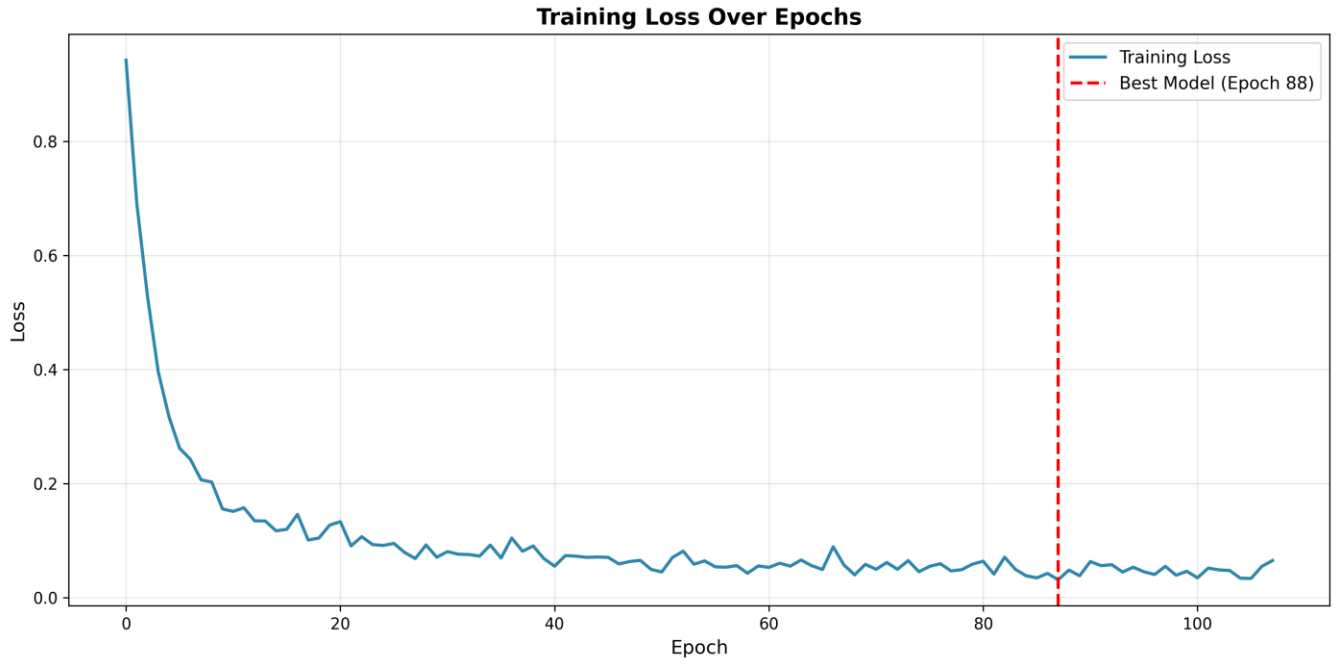

Figure 2 -- Training loss over 108 epochs. Rapid decrease in early epochs followed by 0.03-0.05. Red dashed line marks the best model saved at Epoch 88.

6. Results

6.1 Generated Images at Different Epochs

Generated samples were saved every 10 epochs to track training progress. The following samples show generation quality at epochs 10, 50, and 100.

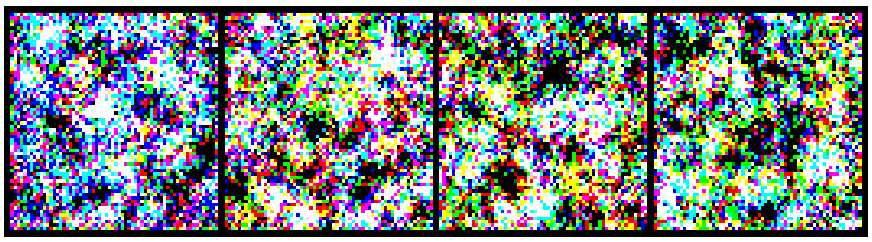

Figure 3 -- Generated samples at Epoch 10. Outputs are largely noisy with no structure.

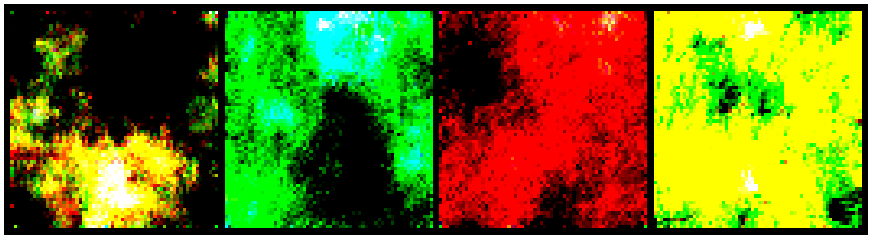

Figure 4 -- Generated samples at Epoch 50. Color blobs begin to emerge showing early learning.

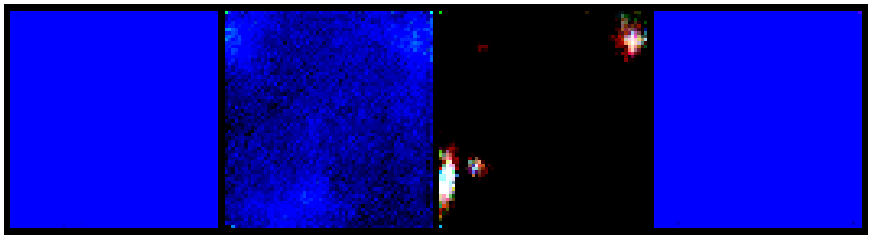

Figure 5 -- Generated samples at Epoch 100. More defined color regions, model has learned dominant color distributions.

6.2 Final Generated Samples

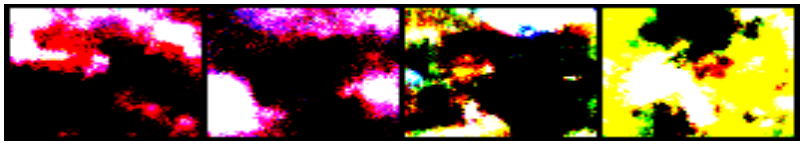

Figure 6 -- Final generated images (2x2 grid) from the best saved model. Images show learned color distributions from the animal dataset.

6.3 Reconstruction Comparison

A reconstruction test was performed by running a real image through the forward process (adding noise) and then running the reverse process to attempt recovery. Results are shown below.

Figure 7a -- Original bear image from dataset (64x64).

Figure 7b -- Reconstructed image after reverse diffusion. Color tones of the background (greens and browns) are partially recovered, but fine structural detail is lost.

The reconstructed image retains background color semantics (greens and browns matching the bear's natural environment) but lacks structural sharpness. This is consistent with expected behavior for a model trained on a small dataset. The non-random output confirms the reverse diffusion pipeline is functioning correctly.

7. Problems and Solutions

Problem	Cause	Solution
Blurry generated images	Only 100 training images	Documented as dataset limitation; model still learns color semantics
Early stopping at epoch 108	Loss plateau with patience=20	Best model at epoch 88 saved and used for inference

8. Conclusion

A complete pipeline was successfully implemented from scratch using PyTorch, including the DataLoader, forward diffusion process using the closed-form noise equation, U-Net denoising model with time conditioning, custom MSE loss, and reverse sampling. The model demonstrated successful learning as evidenced by loss reduction from 0.94 to 0.03 and non-random generated outputs that reflect color distributions of the training data. Generation quality was limited by the

small dataset size (100 images) and training duration (108 epochs). With a larger dataset and extended training, the model architecture is capable of producing sharper and more semantically meaningful images.